

0. Setup obrigatório (primeira célula)

```
import pandas as pd

postos = pd.read_csv(

"/Users/arthurmalucelli/FGV/Programacao/Aulas/05.26/postos_sp_2025.csv",
    sep=";", decimal=",",
)

distrib = pd.read_csv(

"/Users/arthurmalucelli/FGV/Programacao/Aulas/05.26/distribui
cao_municipios_sp_2025.csv",
    sep=";", decimal=",",
)

postos["CNPJ"] = postos["CNPJ"].str.strip()
postos["Mes"] = postos["Data da Coleta"].str[6:10] + "-" +
postos["Data da Coleta"].str[3:5]

print(postos.shape, distrib.shape)
postos.head()
```

CSV brasileiro: sempre sep=";" e decimal=",". Sem isso, parsing quebra ou número vira string.

1. Inspeção rápida

Comando	O que mostra
df.head(n) / tail(n)	n primeiras/últimas linhas (default 5)
df.sample(n)	n linhas aleatórias
df.shape	(linhas, colunas), sem ()
df.info()	colunas, tipos, não-nulos
df.columns / dtypes	nomes / tipos
df.describe()	count, mean, std, min, quartis, max
df.describe(include="all")	inclui colunas de texto
len(df)	número de linhas (igual shape[0])

2. Seleção de coluna

```
df["c"]          # Series (1D)
df[["c1", "c2"]] # DataFrame (várias colunas, ordem da
lista)
df[["c"]]        # DataFrame com 1 coluna (≠ Series)
```

1 colchete vs 2: um vira Series, dois viram DataFrame. Pra reordenar colunas, usa 2.

3. Filtro lógico (linha)

```
# Sempre 2 etapas mentais:
cond = df["Produto"] == "GASOLINA" # Series booleana
df[cond]                            # DataFrame filtrado

# Ou direto:
df[df["Produto"] == "GASOLINA"]
df[df["Valor de Venda"] > 6]
df[df["Bandeira"] != "BRANCA"]
df[df["Bandeira"].isin(["VIBRA", "RAIZEN", "IPIRANGA"])]
len(df[df["Produto"] == "GASOLINA"])
```

Op	Significado
== !=	igual, diferente
> < >= <=	comparações numéricas
.isin([...])	está na lista

4. Combinando condições

```
# NUNCA usar and/or/not em pandas
df[(df["Produto"]=="GASOLINA") & (df["Valor de Venda"]>6)]
# AND
df[(df["Bandeira"]=="VIBRA") | (df["Bandeira"]=="RAIZEN")]
# OR
df[~(df["Bandeira"]=="BRANCA")]
# NOT

# Versão legível (recomendada na prova):
c1 = df["Produto"] == "GASOLINA"
c2 = df["Valor de Venda"] > 6
c3 = df["Município"] == "SAO PAULO"
df[c1 & c2 & c3]
```

Pegadinha #1: cada condição entre parênteses. df[df["x"]==1 & df["y"]>2] sem parênteses dá erro silencioso de precedência.

5. loc / iloc (filtro + coluna)

```
df.loc[cond, "Valor de Venda"] # linhas
onde cond, só Valor
df.loc[cond, ["Revenda", "Valor de Venda"]] # várias
colunas
df.loc[df["Produto"]=="GASOLINA", "Valor de Venda"].mean()

df.iloc[0]          # linha 0 (por posição)
df.iloc[0:5, 0:3]   # 5 linhas, 3 colunas (FIM EXCLUSIVO)
df.loc[5:10]        # linhas 5..10 (FIM INCLUSIVO)
```

	loc	iloc
Usa	rótulo (nome)	posição (número)
Fim slice	inclusivo	exclusivo

6. Strings na coluna (.str)

```
df["c"].str.upper() / .lower() / .title() / .swapcase()
df["c"].str.strip() # tira espaços das
pontas
df["c"].str.len()   # tamanho
df["c"].str.contains("POSTO") # booleana (filtro!)
df["c"].str.startswith("AUTO") / .endswith("LTDA")
df["c"].str.replace("LTDA", "") # SUBSTRING (≠ replace
puro)
df["c"].str[0]      .str[-3:] .str[6:10] # slice por
posição
df["c"].str.split() # quebra por espaço,
vira lista
df["c"].str.split(expand=True) # vira DataFrame
df["c"].str.split().str[-1]   # último elemento (ex:
sobrenome)

# Encadeamento (lê esq-dir):
df["c"].str.upper().str.replace("LTDA", "").str.strip()
```

Sem .str: df["c"].upper() dá AttributeError. Sempre .str antes de função de string.

7. replace vs str.replace

Forma	Compara o quê
col.replace("SP", "Sao Paulo")	valor inteiro da célula
col.str.replace("SP", "BA")	substring dentro do texto

```
df["Estado"] = df["Estado"].replace("SP", "Sao Paulo")
df["Bandeira"] =
df["Bandeira"].replace({"BRANCA": "SEM", "VIBRA": "BR"})
```

8. Criar / atualizar coluna

```
df["nova"] = df["a"] + df["b"] # cria à direita
df["a"]    = df["a"] * 1.1     # substitui
valores
df["Pais"] = "Brasil"          # constante
df["Cidade_U"] = df["Município"].str.upper()
```

Pegadinha #2: df = df["c"].str.lower() DESTRÓI o DataFrame (vira Series). Pra atualizar coluna, esquerda tem que ser df["c"], não df.

9. Conversão de tipo (astype)

```
df["c"] = df["c"].astype(float)
df["c"] = df["c"].astype(str)
df["c"] = df["c"].astype(int)

# Se Valor veio como string (decimal não pegou):
df["Valor de Venda"] = (
    df["Valor de Venda"]
    .str.replace(",", ".")
    .astype(float)
)

# Concatenar string+número: "C" + df["id"] falha. Converte primeiro: "C"
+ df["id"].astype(str).
```

10. Estatísticas numéricas

```
df["c"].mean()      .median()    .std()    .var()
df["c"].min()       .max()       .sum()
df["c"].count()     # não-nulos
df["c"].quantile(0.25) .quantile(0.75)
df["c"].describe()  # tudo de uma vez

# Em DataFrame inteiro:
df.describe()        # só numéricas
df.describe(include="all") # também texto
```

Sem (): df["c"].mean retorna *bound method*, não o número. Erro silencioso clássico.

11. value_counts / unique / mode

```
df["c"].value_counts()      # contagem
decrecente
df["c"].value_counts(normalize=True) # proporção (0..1)
df["c"].value_counts(dropna=False)  # inclui NaN
df["c"].value_counts().head(3)      # top 3
df["c"].value_counts()["VIBRA"]     # contagem de 1 valor

df["c"].unique()             # array dos distintos
df["c"].nunique()            # quantos distintos (1 número)
df["c"].mode()               # valor(es) mais frequente(s)
```

Pergunta	Método
Que valores existem?	unique
Quantos distintos?	nunique
Quantos de cada?	value_counts
Qual mais frequente?	mode ou vc().index[0]

12. Ordenação e duplicatas

```
df.sort_values("c")      # crescente, NÃO
persiste
df.sort_values("c", ascending=False)
df = df.sort_values("c") # PERSISTE
(reatribui)

df.drop_duplicates()      # linhas 100%
iguais
df.drop_duplicates(subset=["CNPJ"]) # único por CNPJ
```

13. Nulos (NaN)

```
df.isnull().sum()      # quantos nulos por coluna
df.dropna()            # remove linhas com qualquer nulo
df.dropna(subset=["c"]) # só pelo critério da coluna c
df.fillna(0)           # preenche com 0
df["c"].fillna("nd")   # preenche coluna específica
```

14. groupby (a operação mais importante)

```
# Estrutura: df.groupby(CRITÉRIO)[COLUNA].FUNÇÃO()
df.groupby("Bandeira")["Valor de Venda"].mean()
df.groupby("Produto")["Valor de Venda"].max()
df.groupby("Município")["CNPJ"].nunique()    # postos
distintos por cidade
df.groupby("Bandeira").size()                # qtd linhas por
grupo

# Múltiplos critérios (lista):
df.groupby(["Município","Produto"])["Valor de Venda"].mean()

# Critério como coluna em vez de índice:
df.groupby("Bandeira", as_index=False)["Valor de
Venda"].mean()

# Filtro + groupby:
df[df["Produto"]=="GASOLINA"].groupby("Município")["Valor de
Venda"].mean()
```

Função	Faz
.mean()	média
.sum() .median()	soma, mediana
.min() .max() .std()	extremos, dispersão
.size()	qtd linhas no grupo
.count()	não-nulos numa coluna
.nunique()	distintos numa coluna
.first() .last()	primeiro/último valor

15. agg (várias estatísticas)

```
# Várias funções pra mesma coluna:
df.groupby("Bandeira")["Valor de
Venda"].agg(["mean","min","max","std"])

# Diferentes funções pra diferentes colunas:
df.groupby("Bandeira").agg({
    "Valor de Venda": "mean",
    "CNPJ": "nunique",
})

# Combinado:
df.groupby("Bandeira").agg({
    "Valor de Venda": ["mean","min","max"],
    "CNPJ": "nunique",
})
```

16. Ranking, top N, idxmax

```
df.groupby("Bandeira")["Valor de
Venda"].mean().sort_values(ascending=False)
df.groupby("Município")["Valor de
Venda"].mean().sort_values(ascending=False).head(5)

# Rótulo do MAIOR/MENOR (não o valor):
df.groupby("Município")["Valor de Venda"].mean().idxmax() #
nome do município
df.groupby("Município")["Valor de Venda"].mean().max()    #
valor da média
df["Valor de Venda"].idxmax()    # índice da linha com maior
valor
```

17. pivot_table (= tabela dinâmica)

```
df.pivot_table(
    index="Município",      # rows
    columns="Produto",      # columns
    values="Valor de Venda", # values
    aggfunc="mean",         # função
    margins=True,          # linha/col de total
)
```

Equivale a groupby(["Município","Produto"])["Valor de Venda"].mean().unstack(). Use pivot_table se for mais natural.

18. unstack / stack

```
# Resultado de groupby multi-key fica vertical:
df.groupby(["Bandeira","Produto"])["Valor de Venda"].mean()

# .unstack() joga último nível pra coluna (vira matriz):
df.groupby(["Bandeira","Produto"])["Valor de
Venda"].mean().unstack()

# Escolhe nível:
.unstack(0) ou .unstack("Bandeira")
```

19. merge (junção, PROCV++)

```
pd.merge(left, right, on="chave", how="inner")
pd.merge(L, R, on=["k1","k2","k3"], how="inner")    # chave
múltipla
pd.merge(L, R, left_on="A", right_on="B", how="left") #
nomes diferentes

# Sintaxe método (igual):
L.merge(R, on="chave", how="inner")
```

how	Linhas resultantes
inner (default)	só chaves presentes nos dois
outer	união, faltante = NaN
left	tudo do left + match do right
right	tudo do right + match do left

Pegadinha #3: chave incompleta → produto cartesiano (DataFrame explode pra milhões). Se shape ficar gigante depois do merge, falta coluna no on.

20. Caso da prova: postos + distrib

```
# 1. Cria chave Mes em postos (formato YYYY-MM):
postos["Mes"] = (postos["Data da Coleta"].str[6:10] + "-"
    + postos["Data da Coleta"].str[3:5])

# 2. Junta pela chave tripla:
juntos = pd.merge(
    postos, distrib,
    on=["Município","Produto","Mes"],
    how="inner",
)

# 3. Calcula margem:
juntos["margem"] = juntos["Valor de Venda"] -
juntos["Preco_Distribuicao"]

# 4. Margem média por bandeira (ranking):
juntos.groupby("Bandeira")
["margem"].mean().sort_values(ascending=False)
```

21. plot (gráficos rápidos)

```
df.plot(x="data", y="quantidade", kind="line", marker="o",
    title="X", xlabel="data", ylabel="qtd")

# kind: "line", "bar", "barh", "pie", "hist", "box"
# Múltiplas séries: y=["col1","col2"]
# Resultado de groupby também tem .plot:
df.groupby("Mes")["Valor de Venda"].mean().plot(kind="line")
```

22. Cenários típicos (com solução)

QUANTOS POSTOS DISTINTOS EM SP CAPITAL

```
df[df["Município"]=="SAO PAULO"]["CNPJ"].nunique()
```

TOP 5 CIDADES COM GASOLINA MAIS CARA (MÉDIA)

```
(df[df["Produto"]=="GASOLINA"]
 .groupby("Município")["Valor de Venda"]
 .mean()
 .sort_values(ascending=False)
 .head(5))
```

BANDEIRA MAIS FREQUENTE EM SP CAPITAL

```
df[df["Município"]=="SAO PAULO"]
["Bandeira"].value_counts().head(1)
```

MUNICÍPIO COM DIESEL MAIS CARO (NOME)

```
(df[df["Produto"]=="DIESEL S10"]
 .groupby("Município")["Valor de Venda"].mean().idxmax())
```

PREÇO MÉDIO MÊS A MÊS DE GASOLINA EM SP

```
sp = df[(df["Produto"]=="GASOLINA") & (df["Município"]=="SAO PAULO")].copy()
sp["Mes"] = sp["Data da Coleta"].str[6:10] + "-" + sp["Data da Coleta"].str[3:5]
sp.groupby("Mes")["Valor de Venda"].mean()
```

POSTOS CUJA REVENDA CONTÉM "AUTO POSTO" E BANDEIRA VIBRA

```
c1 = df["Revenda"].str.contains("AUTO POSTO")
c2 = df["Bandeira"]=="VIBRA"
df[c1 & c2]
```

MUNICÍPIOS ONDE GASOLINA MÁXIMA PASSOU DE 7

```
g = df[df["Produto"]=="GASOLINA"]
m = g.groupby("Município")["Valor de Venda"].max()
m[m > 7]
```

DISTRIBUIÇÃO COMPLETA POR PRODUTO

```
df.groupby("Produto")["Valor de Venda"].describe()
```

QUANTAS BANDEIRAS OPERAM EM CADA CIDADE (TOP 10)

```
df.groupby("Município")
["Bandeira"].nunique().sort_values(ascending=False).head(10)
```

23. Erros típicos e diagnóstico

Mensagem	Causa provável
KeyError: 'X'	coluna não existe (case, espaço, acento), ou df virou Series
TypeError: unsupported operand	operação aritmética em coluna string. Usar astype(float)
AttributeError: 'Series' has no 'upper'	esqueceu .str
'method' object does not support item assignment	usou [em vez de (em método
'float' object does not support item assignment	atribuição em cadeia a = b = expr com efeito colateral
ParserError tokenizing data, expected N saw M	faltou sep=";" no read_csv
FileNotFoundError	caminho relativo errado; usa absoluto
SettingWithCopyWarning	atribuição em DF filtrado; usa .copy() ao criar subset
DataFrame explodiu (shape gigante depois de merge)	chave incompleta no on
Output <bound method ...>	esqueceu () ao chamar função

Dica do prof: erro grande e vermelho, lê a última linha do traceback. É onde o pandas diz o que houve.

24. Mostrar 2 resultados na mesma célula

```
display(df["a"].mean())
display(df["b"].mean()) # Jupyter mostra os 2
```

Sem display, só o último valor da célula aparece.

25. Atalhos mentais (regras-mãe)

- = vs ==: um atribui, dois compara. Erro é clássico.
- Estatística sempre em coluna: df["c"].mean(), não df.mean().
- Reordenar / atualizar coluna: esquerda é df["c"], não df.
- Booleanos: & | ~, cada cond entre parênteses. Nunca and/or/not.
- Slice loc: fim inclusivo. iLoc: fim exclusivo.
- Métodos com (), atributos sem. shape sem, head() com.
- sort_values não persiste sozinho. Reatribui.
- Sempre .str antes de função de string.
- Salva o notebook a cada exercício (Cmd+S).
- Antes de submeter: Kernel → Restart and Run All pra confirmar que tudo roda em ordem.

26. Fluxo dos 100 min

1. Min 1–5: lê todas as perguntas antes de tocar em código. Classifica mental (filtro? groupby? merge?).
2. Min 5–10: roda setup. Confere shape e head. Se Valor veio string, converte (seção 9).
3. Min 10–50: faz as fáceis (value_counts, mean simples, filtro). Trava pontos.
4. Min 50–80: combinadas (filtro+groupby, merge, encadeamento).
5. Min 80–85: Restart and Run All pra garantir consistência.
6. Min 85–100: eClass abre. Representante sobe o.ipynb e responde questionário. Confere nomes dos 3 do grupo.